

COMPILER DESIGN

Infix to Postfix Conversion

Identifying the Suitable Data Structure and Justifying Its Efficiency

Technical Presentation



Why Convert Infix to Postfix?

The compiler evaluates expressions faster without needing precedence rules or parentheses.

Infix

Operator sits between operands — natural for humans, but ambiguous for machines.

A + B * C

Postfix

Operator follows its operands — no parentheses or precedence lookup needed.

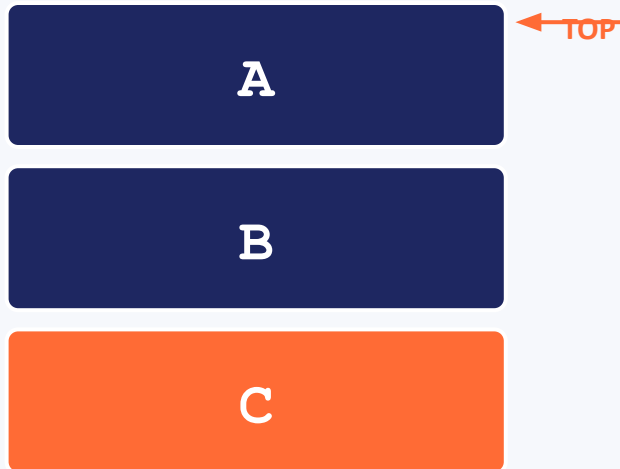
A B C * +

During Compilation

- ✓ Precedence and associativity are resolved once, during conversion
- ✓ Evaluation becomes a single left-to-right scan
- ✓ No backtracking or re-scanning for brackets is required
- ✓ Ideal for machine-level (stack-based) execution

The Stack: A LIFO Structure

Last-In, First-Out — elements are added and removed from the same end.



push (x)

Insert element x on top



pop ()

Remove and return the top element



peek ()

View the top element without removing it

$O(1)$ time for push, pop, and peek — the defining efficiency of a stack.

Why the Stack Fits This Problem

Every rule of the conversion algorithm maps directly onto a stack operation.



Operator Reordering

Operators must be held back and released later — LIFO naturally reverses order of emission.



Precedence Handling

Lower-precedence operators pop higher-precedence ones already on the stack before pushing.



Parenthesis Matching

'(' is pushed and ')' pops until the matching '(' — a textbook LIFO nesting problem.



Single Left-to-Right Pass

Each symbol is pushed or popped at most once, keeping the scan strictly linear.

The Conversion Algorithm

A single left-to-right scan of the infix expression, using one stack for operators.

1

Operand encountered → append directly to output

2

Operator encountered → pop stack while top has $\geq$ precedence, then push

3

'(' encountered → push onto stack

4

')' encountered → pop and output until '(' is found, then discard it

5

End of expression → pop all remaining operators to output

Worked Example

Converting **A + B * C - D**

Symbol	Action	Stack	Output
A	Output operand	(empty)	A
+	Push operator	+	A
B	Output operand	+	A B
*	Push (higher prec.)	+ *	A B
C	Output operand	+ *	A B C
-	Pop *, pop +, push -	-	A B C * +
D	Output operand	-	A B C * + D
end	Pop remaining	(empty)	A B C * + D -

Efficiency: Time & Space

Every symbol is examined once; every operator is pushed and popped at most once.

$O(n)$

Time Complexity

One linear pass over n input symbols; each stack op is $O(1)$.

$O(n)$

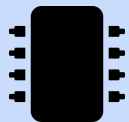
Space Complexity

Worst case: all n symbols are operators (or brackets) held on the stack.

$O(1)$

Per-Operation Cost

push, pop, and peek are constant-time array/linked-list operations.



A stack achieves this in a single pass with no backtracking — an array without stack discipline, or repeated re-scans for precedence, would cost $O(n^2)$ in the worst case.

Why Not Another Structure?

Comparing candidate structures against the demands of the algorithm.

Structure	Access Order	Fit for This Problem
Stack	LIFO	Exact match — precedence & bracket nesting resolve naturally
Queue	FIFO	Cannot reorder operators; wrong operators would emit first
Plain Array	Random	Works only if stack discipline is re-implemented manually
Recursion / Tree	Hierarchical	Suited to parsing into a tree, but adds overhead this task doesn't need



The stack is the only structure that satisfies LIFO ordering, $O(1)$ operations, and single-pass scanning simultaneously.

Conclusion



The Stack (LIFO) is the suitable structure for infix-to-postfix conversion: it matches the operator-reordering and bracket-nesting needs of the algorithm while offering $O(1)$ push/pop and an overall $O(n)$ single-pass conversion.

Where This Applies



Compiler expression evaluation



Calculator engines



Arithmetic parsers & VMs

Key takeaway: matching the data structure's access pattern to the algorithm's needs is what delivers linear-time efficiency.